

Frontend Task

Build the tactical command deck, live telemetry display, map layers, evacuation route view, and critical incident workflow described in [docs/API.md](#).

Task Specification: System Architect & Backend Lead

Role: System Architect, Core Pipeline & Backend Lead

Objective: Architect the central FastAPI service, manage real-time WebSocket telemetry distribution, enforce hydrodynamic rate-of-rise threshold logic, integrate geospatial data layers from the GIS engineer, and provide robust API contracts to unblock the Frontend engineer immediately.

Milestone 1: College Demo Deliverables

Focus: A fully running FastAPI server with mock-first endpoints, an active WebSocket broadcaster, on-demand cloudburst surge injection, and automated state-transition logic.

- **Task 1.1: Environment, Scaffolding & API Contract Setup**
- Set up the isolated Python 3.11 environment ([expenv](#)) and project directory:

```
backend/
├── app/
│   ├── api/routes/      # telemetry.py, villages.py, simulation.py
│   ├── core/           # config.py, state.py
│   ├── models/         # Pydantic schemas
│   ├── services/       # hydro_rules.py, stream_manager.py
│   └── main.py
├── data/               # static geojson assets from GIS lead
├── requirements.txt
└── run.sh
```

- Implement Pydantic data schemas matching [docs/API.md](#):
- **TelemetryPayload:** `timestamp`, `sensor_id`, `rainfall_mm_hr`, `water_level_cm`, `rate_of_rise_cm_min`, `soil_moisture_pct`, `status`.
- **VillageStatus:** `id`, `name`, `risk_level`, `risk_score`, `lead_time_minutes`, `population_at_risk`, `primary_driver`.
- **SimulationRequest:** `intensity`, `target_basin`.
- Provide stub/mock responses for `GET /api/villages` and `GET /api/evacuation/{village_id}` so the Frontend engineer can immediately fetch valid payloads.

- **Task 1.2: Real-Time WebSocket Telemetry Engine**

- Build an async WebSocket endpoint at `WS /ws/telemetry`.
- Implement a background tick worker (`asyncio.sleep(2)`) broadcasting baseline readings:
- Rainfall: 12–15 mm/hr
- Water level: 108–112 cm
- $\frac{dh}{dt}$: +0.1– + 0.3 cm/min
- State: **NORMAL**
- Maintain a thread-safe in-memory state engine to store the latest 10 readings for sliding-window calculations.

- **Task 1.3: Hydrodynamic Surge & Rate-of-Rise (dh/dt) Rules Engine**

- Implement rate-of-rise acceleration monitoring:

$$\frac{dh}{dt} = \frac{h_t - h_{t-1}}{\Delta t}$$

- Define operational thresholds:
- $\frac{dh}{dt} < 1.0 \text{ cm/min} \implies$ **NORMAL** (Green)
- $1.0 \leq \frac{dh}{dt} < 2.0 \text{ cm/min} \implies$ **WATCH** (Yellow)
- $\frac{dh}{dt} \geq 2.0 \text{ cm/min} \implies$ **CRITICAL** (Red)
- Calculate estimated peak surge arrival time dynamically based on upstream gauge distance (8.4 km) and propagation speed ($v \approx 3.5\text{--}4.5 \text{ km/h}$):

$$\text{Lead Time} = \frac{\text{Distance}}{\text{Velocity}} \approx 42\text{--}48 \text{ minutes}$$

- **Task 1.4: Cloudburst Injection Trigger (`POST /api/simulate/cloudburst`)**

- Build the demo trigger route. When invoked via POST or Swagger UI:
 1. Override background ticker data with surge parameters: rainfall jumps to 85 mm/hr, water depth spikes rapidly (118 $\rightarrow$ 135 $\rightarrow$ 164 cm), and $\frac{dh}{dt}$ spikes to +3.8 cm/min.
 2. Evaluate threshold rules $\implies$ auto-transition **VIL_TILWARA** to **CRITICAL**.
 3. Instantly broadcast the critical payload across active WebSocket connections.

- **Task 1.5: Static Vector Asset Proxy**

- Load `villages.geojson`, `river_mandakini.geojson`, and `evac_route_tilwara.geojson` from `backend/data/` (committed by the GIS lead).
- Expose endpoints `GET /api/geojson/{layer_name}` or serve them as static mount files with enabled CORS headers (`CORSMiddleware`) for the Frontend team.

Focus: Ingesting offline GIS terrain tables, serving the ML classification engine with SHAP explainability, implementing risk-weighted Dijkstra evacuation routing, and adding telemetry sanity filters.

- **Task 2.1: Sensor Telemetry Anomaly & Health Filter**

- Build a pre-processing validation layer to eliminate corrupted IoT inputs before reaching the threshold engine:
- *Stale/Stuck Check*: Flag hardware as **FAULTY_STUCK** if depth variance < 0.1 cm across 20 consecutive ticks during heavy rainfall.
- *Impossible Spike Check*: Discard readings where $|h_t - h_{t-1}| > 50$ cm within a 2-second sampling tick (transducer noise).
- *Timeout Watchdog*: Set village state to **DATA_OFFLINE** if no packet arrives within 15 seconds.

- **Task 2.2: Dual-Horizon Risk Model Integration (6–24h Layer)**

- Merge live weather/soil telemetry with the static terrain metrics generated by the GIS lead (**village_terrain_features.csv**: mean slope, drainage area, TWI).
- Load the pre-trained **xgboost_watershed_risk.json** model serialized by the data track.
- Expose **GET /api/risk/detailed/{village_id}** returning predicted susceptibility probabilities across 4 tiers.

- **Task 2.3: Explainable AI (SHAP) Factor API**

- Compute local SHAP feature contributions for incoming predictions using **shap.TreeExplainer**:
- Extract normalized feature impact:

$$\sum \phi_i = f(x) - E[f(x)]$$

- Format output into a ranked key-value response:

```
{
  "primary_driver": "Extreme upstream rainfall accumulation",
  "factors": [
    { "feature": "3h_accumulated_rainfall", "impact_pct": 38 },
    { "feature": "antecedent_soil_saturation", "impact_pct": 31 },
    { "feature": "topographic_wetness_index", "impact_pct": 21 },
    { "feature": "sub_basin_slope", "impact_pct": 10 }
  ]
}
```

- **Task 2.4: Dynamic Risk-Weighted Evacuation Router**

- Load **rudraprayag_roads.graphml** (provided by the GIS lead) into **networkx**.
- Implement dynamic edge-cost weighting based on flood hazard:

$$\text{Cost}_e = \text{Length}_e \times \text{Hazard Multiplier}_e$$

(where roads in high-risk low-elevation zones receive a penalty multiplier $\times 10$ to ∞).

- Run Dijkstra's shortest-path algorithm from the centroid of `VIL_TILWARA` to `SHELTER_01`.
- Serialize output paths as GeoJSON LineStrings via `GET /api/evacuation/{village_id}`.
- **Task 2.5: Low-Bandwidth SMS / 2G Alert Service**
- Add a notification adapter (`services/alert_dispatcher.py`) integrating Twilio or Fast2SMS.
- Generate a compressed plain-text tactical message format (≤ 140 characters) for local administrative officers:

```
[NDRF ALERT] MANDAKINI BASIN: Flash surge detected (+3.8cm/m). Tilwara sector at risk in 42m. Evacuate via High Road to Inter College.
```

Verification & Quality Checklist

- ☐ CORS middleware explicitly allows frontend local origins (`http://localhost:5173` or `http://localhost:3000`).
- ☐ All WebSocket reconnections handle unexpected client disconnects without worker crashes.
- ☐ End-to-end latency from `POST /api/simulate/cloudburst` to WebSocket broadcast remains under 250 ms.
- ☐ Fully documented interactive Swagger documentation available at `/docs`.